

本科毕业设计（论文）

基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏 设计与实现

学生姓名：（请填写姓名）

学 号：（请填写学号）

专业班级： 计算机科学与技术 （请填写班级）

指导教师：（请填写指导教师）

2026 年 6 月

学位论文原创性声明

本人所提交的学位论文《基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏设计与实现》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。本声明的法律后果由本人承担。

论文作者（签名）：_____ 指导教师确认（签名）：_____
年 月 日 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）：_____ 指导教师（签名）：_____
年 月 日 年 月 日

摘 要

随着 HTML5 与 WebGL 技术的成熟，浏览器端已能够承载中等复杂度的实时动作游戏。Roguelite 品类以单局随机性、局外元进度与构筑深度为核心乐趣，对架构可扩展性、数据驱动能力与运行时性能提出了较高要求。本文以毕业设计项目 *Survivor And Fight* 为对象，在 LayaAir 3.x 引擎与 TypeScript 技术栈上，设计并实现了一套面向 2D 俯视角生存战斗的完整游戏系统。

系统采用轻量级实体组件系统（ECS）作为玩法内核，将位置、速度、属性、技能、操控等数据与逻辑解耦为组件与系统，并通过分组调度与命名筛选器支撑玩家、怪物及可操控实体的批量更新。战斗层实现配表驱动的技能与效果执行链，支持子弹发射、穿透、分裂、连锁等组合效果；子弹与怪物通过对象池减少实例化开销；在实体规模达到阈值时，战斗快照可通过 Web Worker 异步重算，主线程负责采集与应用结果。元游戏层提供开始界面、选关与三大关爬塔式跑图，局内支持经验升级、随机奖励与技能装配界面（MVC 与 UI 栈）。游戏引擎架构理解、物理模拟与模块化设计方面的研究为本文提供了理论依据 [19, 7, 5]。

本文从需求分析、总体架构、模块设计与实现、性能优化与测试验证等方面展开论述。测试表明，在目标平台上系统可稳定运行，对象池与 Worker 卸载对帧率稳定性具有明显改善。

关键词：LayaAir；实体组件系统；Roguelite；数据驱动；Web Worker；对象池；生存战斗

Abstract

With the maturity of HTML5 and WebGL, browsers can host real-time action games of moderate complexity. This thesis presents the design and implementation of *Survivor And Fight*, a 2D top-down survival combat game built on LayaAir 3.x and TypeScript. A lightweight ECS drives gameplay; combat is configuration-driven with object pooling and optional Web Worker offloading. A meta layer provides menus, level selection, and a three-act DAG run map. Results show stable operation with measurable gains from pooling and worker offloading.

Keywords: LayaAir; ECS; Roguelite; data-driven; Web Worker; object pooling; survivor-like combat

目录

摘要	2
Abstract	3
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 游戏引擎架构	1
1.2.2 实体组件系统与玩法架构	2
1.2.3 物理模拟、人群与性能	2
1.3 研究内容与论文结构	2
1.4 竞品与项目定位简要分析	2
1.5 本章小结	3
第二章 相关技术与理论基础	4
2.1 LayaAir 游戏引擎与 TypeScript	4
2.2 实体组件系统	4
2.3 数据驱动与配表	4
2.4 Roguelite 与跑图	5
2.5 Web 性能与 UI 架构	5
2.6 Unity DOTS 与轻量 ECS 对比	5
2.7 本章小结	5
第三章 系统需求分析	6
3.1 可行性分析	6
3.1.1 技术可行性	6

3.1.2	经济与社会可行性	6
3.2	功能性需求	6
3.2.1	用例：局内战斗	6
3.2.2	用例：元游戏进入战斗	7
3.3	非功能性需求	7
3.4	需求优先级	7
3.5	运行环境	8
3.6	本章小结	8
第四章	系统总体设计	9
4.1	设计原则	9
4.2	系统分层架构	9
4.3	ECS 调度与战斗数据流	10
4.4	元游戏与配表模型	10
4.5	部署与容错	10
4.6	本章小结	10
第五章	核心模块详细设计与实现	11
5.1	ECS 核心模块	11
5.2	属性、移动与筛选器	11
5.3	技能与效果执行	11
5.4	子弹系统与碰撞	11
5.5	怪物系统	12
5.6	进度、元游戏与跑图	12
5.7	技能装配 UI	12
5.8	配置加载	12
5.9	典型场景剖析	13
5.10	本章小结	13
第六章	性能优化与工程实践	14
6.1	性能目标与瓶颈	14
6.2	对象池	14
6.3	Web Worker 卸载	14
6.4	其它优化	14

6.5	绿色计算与兼容性	15
6.6	本章小结	15
第七章	系统测试与验证	16
7.1	测试策略	16
7.2	单元验证	16
7.3	功能测试	16
7.4	性能测试	17
7.5	跑图与 Worker 一致性	17
7.6	测试结论	17
7.7	本章小结	17
第八章	总结与展望	18
8.1	工作总结	18
8.2	不足与展望	18
8.3	社会、健康与文化影响	18
8.4	本章小结	19
致谢		20
附录 A	名词术语及缩略词	24
附录 B	核心配表字段示例（节选）	25

插图

4.1 系统分层架构示意	9
------------------------	---

表格

2.1	重型 ECS 与项目轻量 ECS 对比	5
3.1	主要功能性需求一览	7
3.2	需求优先级 (MoSCoW)	7
5.1	效果类型与执行器映射	12
7.1	功能测试用例摘要	16
7.2	性能对比实验 (待填入实测数据)	17

第一章 绪论

1.1 研究背景与意义

移动互联网与浏览器平台的普及，使“打开即玩”的 H5 游戏成为独立游戏与教学实践的重要载体。相较于原生客户端，H5 游戏在分发成本、跨平台兼容与快速迭代方面具有优势；与此同时，JavaScript 单线程事件循环、垃圾回收抖动以及渲染管线约束，也对实时战斗类游戏的架构设计提出了更高要求。Roguelite 品类以自动攻击、大量敌人、局内升级构筑为核心循环，对数据驱动能力、同屏实体规模与元游戏流程编排提出了系统化工程需求。

本课题来源于本科毕业设计项目 **Survivor And Fight**，目标是在 LayaAir 3.x 上实现 2D 俯视角生存战斗游戏原型，覆盖主菜单、选关、爬塔式跑图、局内战斗、升级奖励与技能装配等完整闭环，并探索 ECS 架构、配表驱动技能、对象池与 Web Worker 等性能手段。游戏引擎是商业游戏开发的基础工具，其架构复杂、子系统耦合度高，开发者常面临可维护性与演化性问题 [19, 7, 2]。在 Unity 等引擎上，物理模拟已用于人群疏散等场景验证 [5, 3]，说明引擎层物理与逻辑分离、模块化设计的价值。本课题在 H5 环境下实践类似思想，具有工程与教学双重意义。

1.2 国内外研究现状

1.2.1 游戏引擎架构

SyDRA 方法通过对多款开源游戏引擎进行子系统依赖恢复，生成可比较的架构模型，帮助开发者完成架构理解与影响分析任务 [19]。Gregory 系统论述了现代游戏引擎的渲染、资源、场景图与脚本等子系统划分 [7]。Munro 等对 Quake 系列引擎的架构研究、Agrahari 对 Unreal 内部结构的分析，均为理解引擎模块边界提供了参考 [12, 1]。Goslin 介绍的 Panda3D 引擎体现了脚本与渲染协同的典型结构 [6]。Ullmann 等进一步可视化游戏引擎子系统耦合模式，指出过度耦合与目录嵌套是常见演化问

题 [18]。

1.2.2 实体组件系统与玩法架构

实体组件系统（ECS）将对象拆为实体标识、纯数据组件与无状态系统，有利于组合式扩展与逻辑批量处理。Unity DOTS 代表“面向数据”的极致路线；教学与中小型项目常采用 Map 存储组件、单线程顺序更新的轻量实现，在清晰度与性能间折中。Survivor-like 玩法强调波次刷怪与升级曲线；Roguelite 常采用 Slay the Spire 式 DAG 跑图组织关卡推进。

1.2.3 物理模拟、人群与性能

Reynolds 的 Boids 模型、Thalmann 与 Pelechano 的人群仿真工作，为大量代理体运动提供了经典范式 [16, 17, 14]。Cantrell 等在 Unity 中实现基于物理压力的人群疏散模型并完成验证 [5]；Bott 等使用 Unreal 开发工具实现物理驱动的人群移动 [3]。McKenzie 等将人群行为建模与游戏技术结合用于仿真 [10]。上述研究对本项目怪物追逐、分离力与碰撞简化具有借鉴意义。H5 侧常见优化包括对象池、减少每帧分配、Web Worker 卸载重计算等。

1.3 研究内容与论文结构

本文主要研究内容包括：（1）基于 LayaAir 与 TypeScript 的总体架构；（2）轻量 ECS 核心与玩法系统；（3）配表驱动技能、子弹与碰撞；（4）元游戏流程与 DAG 跑图；（5）MVC 与 UI 栈；（6）对象池与 Worker 性能优化；（7）测试验证及社会影响分析。

全文共分八章：第二章介绍相关技术；第三章给出需求分析；第四章描述总体设计；第五章详述模块实现；第六章讨论性能优化；第七章说明测试；第八章总结与展望。

1.4 竞品与项目定位简要分析

《吸血鬼幸存者》验证了 Survivor-like 品类的市场可行性；《杀戮尖塔》体现了 DAG 跑图的路线决策价值。本项目差异在于采用 LayaAir H5 技术栈、轻量 ECS 与 JSON 配表驱动，面向毕业设计规模的完整可运行原型，而非商业级内容量。

1.5 本章小结

本章阐述了课题背景、国内外相关研究及论文结构，为后续章节奠定理论基础。

第二章 相关技术与理论基础

2.1 LayaAir 游戏引擎与 TypeScript

LayaAir 是面向 HTML5 的跨平台游戏引擎，支持 2D、3D 与 UI。本项目使用 LayaAir 3.x，脚本以 TypeScript 编写，通过 Laya.Script 挂载至 Area2D 子节点，避免不必要的 3D 渲染管线开销。主循环由 `Laya.timer.frameLoop` 驱动，`EcsWorld.update(deltaTime)` 在每帧调用，实现逻辑与渲染解耦。商业引擎手册与架构文献均强调主循环、资源管理与场景更新的一致性 [7, 20]。

TypeScript 提供静态类型与模块化，静态常量集中于 `src/defines/` 并由 `index.ts` 统一导出，符合可维护性设计原则 [4]。

2.2 实体组件系统

ECS 包含 Entity（标识）、Component（数据）、System（逻辑）。本项目 `EntityId` 为数值类型，`ComponentStore` 按类型存储，`EcsWorld` 提供统一 `update` 入口。系统按 `input/logic/physics/render` 分组排序。玩法层包括 `PlayerTag/MonsterTag`、`MovementSystem`、`AttributeSystem`（含可溯源 `modifier`）、`SkillSystem`、`ControlSystem` 及 `FilterRegistry` 命名筛选器。该划分与 Gregory 所述“引擎子系统 + 游戏逻辑层”分层思想一致 [7]，但实现规模适配教学项目。

2.3 数据驱动与配表

策划参数外置为 JSON 表，由 `tables.registry.json` 注册，`ConfigBootstrap.ensureGameCon` 加载。包含 `Character`、`Bullet`、`Skill`、`SkillEffect` 等表。效果类型包括 `bullet`、`modifier_split`、`modifier_chain`、`modifier_pierce`、`direct_damage`，由 `EffectExecutor` 分发。`FormulaParser` 解析伤害公式；`Targeting` 支持自动索敌与指向输入。

2.4 Roguelite 与跑图

每局生成三个大关，每关为多层 DAG，节点含休息、战斗、Boss、宝藏等类型，RunMapGenerator 连接行际边并校验 Boss 可达。SeededRng 支持同种子复现。Helbing 等关于恐慌人群的研究提示：密集交互场景需关注分离与碰撞稳定性 [9]，与本项目怪物拥挤时的分离力设计相关。

2.5 Web 性能与 UI 架构

对象池 (BulletPool、MonsterPool) 减少实例化；CombatDataBridge 在实体数达标时将战斗快照提交 Web Worker；BulletHitTest 使用距离平方比较。GameSession.paused 在技能面板打开时暂停战斗逻辑。

UI 采用 MVC：UiControllerBase 定义生命周期，UIStackManager 管理全屏路由。Heijstek 等对比了架构描述的图文表示对理解效率的影响 [8]，本项目以代码模块划分配合 UI 栈，降低界面跳转歧义。

2.6 Unity DOTS 与轻量 ECS 对比

表 2.1: 重型 ECS 与项目轻量 ECS 对比

维度	Unity DOTS 等	本项目
并行	Job System 多线程	单线程；战斗片段可选 Worker
存储	Archetype/Chunk	按组件类型 Map
引擎绑定	Entities Graphics	ViewComponent 绑 Laya 节点
适用规模	数万实体级	数百实体级

2.7 本章小结

本章综述了 LayaAir、ECS、配表、Roguelite、物理人群仿真相关技术及 UI 架构，并引用了游戏引擎架构与 Unity 物理仿真方面的代表性文献 [19, 5, 7]。

第三章 系统需求分析

3.1 可行性分析

3.1.1 技术可行性

项目选用 LayaAir 3.x 与 TypeScript，已实现 ECS 核心、战斗 Demo、元菜单、跑图生成、技能装配 UI 等模块。引擎支持 2D 预制体、帧循环与键盘输入；浏览器支持 Web Worker。SyDRA 等研究表明，理解引擎子系统结构有助于降低维护成本 [19]；Cantrell 等验证了商业游戏引擎用于复杂物理仿真的可行性 [5]。配表经同步脚本发布至运行目录，技术路线可行。

3.1.2 经济与社会可行性

作为毕业设计原型，主要成本为开发与测试人力。游戏为奇幻战斗题材，须控制暴力表现并提示合理游戏时长；跑图中的休息节点提供节奏缓冲。

3.2 功能性需求

主要功能需求如表 3.1 所示。

3.2.1 用例：局内战斗

参与者：玩家。**前置条件：**配表已加载，SimpleEcsDemo.init() 完成。**主成功场景：**(1) 输入移动；(2) 自动施法；(3) 子弹碰撞、穿透/分裂/连锁；(4) 击杀获经验并升级；(5) Tab 打开技能面板暂停并装配。**扩展：**玩家死亡显示重启面板。

表 3.1: 主要功能性需求一览

模块	需求描述
ECS 核心 玩法实体	创建/销毁实体；按类型存取组件；系统分组更新 玩家/怪物标签；移动、视图同步；hp/maxHp 与 modifier
技能战斗 怪物系统 进度系统 元游戏 跑图	多 effect 技能；索敌；冷却；子弹表驱动伤害与穿透 波次刷怪；追逐；接触伤害；离屏回收 经验、升级、随机奖励 开始/选关；进入战斗；生存计时胜利 三大关 DAG；节点类型；种子可复现
技能装配 UI 配置	Tab 暂停；三装备栏；拖拽装配 JSON 表注册加载；缺失时默认值与日志

3.2.2 用例：元游戏进入战斗

用户在 MetaFlowController 选择关卡或跑图节点，触发 onEnterCombat，显示战斗场景、初始化 Demo、启动生存计时（Boss 关使用更长胜利时间）。

3.3 非功能性需求

性能：目标 60 FPS；同屏怪物受常量与波次控制；配合对象池与 Worker。可维护性：模块边界清晰；静态配置集中于 defines。兼容性：优先 Chromium；Worker 不可用时主线程回退 [5]。

3.4 需求优先级

表 3.2: 需求优先级（MoSCoW）

级别	需求
Must	ECS 战斗、子弹碰撞、移动、刷怪、配表
Must	主菜单进战斗、生存胜利计时
Should	跑图、技能装配、升级奖励、对象池与 Worker
Could	法杖三槽完整 UI、全量 Effect 独立特效
Won't	联机、账号、支付（本期）

3.5 运行环境

建议配置：四核 CPU、8GB 内存、支持 WebGL2 的浏览器；开发环境为 LayaAir IDE 3.x、TypeScript、Node.js 工具链。

3.6 本章小结

本章论证了可行性，列出功能与非功能需求及典型用例，下一章给出总体设计。

第四章 系统总体设计

4.1 设计原则

- (1) 数据与逻辑分离：组件存数据，系统执逻辑；配表驱动参数 [7]。
- (2) 单一职责：BulletSystem 负责子弹与碰撞，EffectExecutor 负责效果语义映射。
- (3) 可验证：核心模块提供单元验证脚本（公式解析、ECS 核心、属性修饰器）。
- (4) 渐进复杂度：轻量 ECS；Worker 仅在规模达标时启用。
- (5) 低耦合：参考游戏引擎子系统解耦经验，避免 UI 直接操作战斗节点 [18]。

4.2 系统分层架构

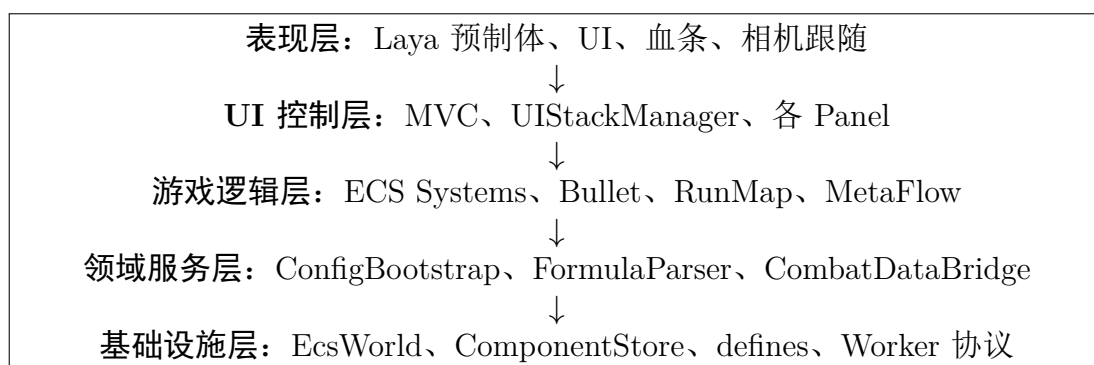


图 4.1: 系统分层架构示意

Main.ts 初始化输入、构造 SimpleEcsDemo,按配置进入元菜单或战斗,在 frameLoop 中更新逻辑与生存计时。

4.3 ECS 调度与战斗数据流

系统分组：input（操控）、logic（属性、移动、技能、刷怪、经验等）、render（视图、血条、HUD）。GameSession.paused 控制战斗暂停。

战斗帧流程：CombatDataPrepareSystem 采集快照 → 可选 Worker 执行 processCombatFrame → 主线程应用追逐/分离结果 → BulletSystem 更新碰撞。该设计与 Cantrell 等将重计算置于引擎逻辑管线、渲染与交互分离的思路一致 [5, 3]。

4.4 元游戏与配表模型

MetaFlowController 协调界面与战斗入口；RunMapState 维护 DAG 进度。逻辑数据关系：Character、Skill 1:N SkillEffect，SkillEffect 可关联 Bullet；跑图节点通过边构成 DAG。

4.5 部署与容错

H5 静态发布至 bin/；无服务端数据库。配表缺失时使用默认常量；Worker 失败回退主线程；跑图生成失败使用保底图；预制体加载失败可用占位碰撞体调试。

4.6 本章小结

本章给出分层架构、ECS 调度、战斗数据流与元游戏设计，下一章详述模块实现。

第五章 核心模块详细设计与实现

本章结合 `src/` 实现，对核心模块作详细说明。

5.1 ECS 核心模块

`EntityManager` 使用自增 ID 与空闲列表复用实体。`ComponentStore` 按类型维护 `Map<EntityId, Component>`。`EcsWorld.update` 依次调用已注册系统,按 `input/logic/physics/re` 分组排序, 与 Gregory 所述引擎主循环驱动子系统更新的模式相对应 [7]。

`ViewComponent`、`BodyUIComponent` 绑定 Laya 节点; `ViewSyncSystem` 同步坐标; `BloodBarSyncSystem` 用 `hp/maxHp` 更新 `ProgressBar`。

5.2 属性、移动与筛选器

`AttributeSystem` 支持 `modifier` 按来源增删,供 `Buff` 与升级奖励使用 [4]。`MovementSystem`、`ControlSystem` 处理位移与输入。`FilterRegistry` 提供 `Players`、`Monsters` 等命名筛选,减少重复查询。

5.3 技能与效果执行

`PlayerAutoCastSystem` 对三装备栏维护独立冷却与 `pendingCasts`。`EffectExecutor` 将配表 `effect` 映射为子弹、穿透/分裂/连锁修饰或直伤。修饰器须位于后续 `bullet effect` 之前,形成链式组合语义。

5.4 子弹系统与碰撞

`BulletSystem` 从 `BulletPool` 取节点,更新运动,用 `hitRadiusSq` 做圆碰撞,按阵营过滤后扣血并处理穿透。密集命中场景可参考人群仿真中的局部相互作用与压

表 5.1: 效果类型与执行器映射

effect 字段	行为
bullet	BulletSystem.spawnBulletWithOptions
modifier_split	设置分裂数量
modifier_chain	设置连锁与半径
modifier_pierce	增加穿透
direct_damage	公式直伤

力模型思想 [9, 16]，本项目采用简化半径检测以降低开销。

5.5 怪物系统

MonsterWaveSpawnSystem 在玩家周围环带刷怪；MonsterChaseSystem 追逐并叠加分离力与随机摆动，Worker 可参与数值重算 [5]。MonsterContactDamageSystem 处理接触伤害；MonsterRecycleSystem 与 MonsterPool 回收离屏单位。

5.6 进度、元游戏与跑图

ExperienceSystem、UpgradeRewardSystem 与奖励面板构成成长闭环。MetaFlowController 协调菜单与 onEnterCombat。RunMapGenerator 生成三幕 DAG，校验 Boss 可达，失败时使用保底图。

5.7 技能装配 UI

SkillSelectPanelController 以 Tab 切换面板并设置 GameSession.paused。SkillDragService 长按拖拽，SkillLoadoutSyncSystem 写回战斗实体。UIStackManager 管理全屏路由，生命周期由 UiControllerBase 约定。

5.8 配置加载

ConfigBootstrap 双通道加载 JSON 并 initData 填充 Data，是数据驱动枢纽 [7, 11]。

5.9 典型场景剖析

首次进入战斗： Main 加载配表 → 元菜单选关 → `demo.init()` 创建玩家与系统 → 生存计时启动。

Tab 装配技能： 暂停战斗 → 拖拽更新 `SkillLoadoutState` → 同步至自动施法 → 恢复战斗。

高密度战斗： 实体数达标时 `CombatDataBridge` 启用 Worker，主线程仍处理子弹碰撞与渲染 [5, 3]。

5.10 本章小结

本章详述了 ECS、技能、子弹、怪物、元游戏、UI 与配置等模块实现，下一章讨论性能优化。

第六章 性能优化与工程实践

6.1 性能目标与瓶颈

H5 生存战斗的热点包括：大量实体更新与碰撞、预制体实例化引发 GC、主线程过载。Cantrell 等在 Unity 中实现物理人群模型时，同样关注计算开销与验证方法 [5]；Bott 等使用游戏引擎工具链实现物理驱动人群移动 [3]。本项目在 2D H5 环境下采用对象池、Worker 卸载与逻辑暂停等手段应对类似瓶颈。

6.2 对象池

BulletPool、MonsterPool 按 prefabPath 分桶复用节点，复用前重置状态。将“每帧大量 instantiate/destroy”转为“峰值后复用”，降低 GC 停顿，与引擎资源管理最佳实践一致 [7]。

6.3 Web Worker 卸载

MonsterChaseSystem 的追逐、分离、摆动为纯数值计算，可由 CombatDataBridge 提交 Worker 执行 processCombatFrame，主线程与 Worker 共享同一逻辑文件以保证一致性。实体数不足或 Worker 不可用时回退主线程。子弹碰撞因依赖 Laya 节点仍留主线程，与 McKenzie 等将游戏技术用于仿真但保留引擎交互层的思路类似 [10]。

6.4 其它优化

碰撞：使用距离平方比较。**暂停：**Tab 打开面板时 GameSession.paused 早退战斗系统。**配置：**常量集中于 defines，配表一次加载。**未来：**可探索 archetype 存储与空间划分 broad phase[19]。

6.5 绿色计算与兼容性

降低平均 CPU 占用有利于终端节能。Worker 回退与双通道配表加载提升兼容性。浏览器或引擎升级后须回归 Worker 脚本路径与发布目录。

6.6 本章小结

本章阐述了对象池、Worker、碰撞微优化与配置管理等性能手段，测试见下一章。

第七章 系统测试与验证

7.1 测试策略

测试分为单元验证、集成测试与系统手工测试。仿真与游戏系统须经过验证确认模型可信度 [15, 13]，本项目采用脚本验证、用例表与性能对比相结合。

7.2 单元验证

- `ecsCorePhase1.verify.ts`: 实体与组件、系统顺序；
- `formulaParser.verify.ts`: 公式边界；
- `attributeModifier.verify.ts`: `modifier` 叠加与移除。

7.3 功能测试

表 7.1 列出代表性用例（手工执行）。

表 7.1: 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动并等待加载	技能表加载成功
T-F-02	进入战斗移动	角色与相机正常
T-F-03	怪物接近	追逐、扣血、血条更新
T-F-04	观察自动攻击	子弹命中，穿透/分裂/连锁
T-F-05	Tab 打开技能面板	暂停，可拖拽装配
T-F-06	关闭面板	战斗恢复，新技能生效
T-F-07	升级	奖励面板与应用

编号	步骤	预期结果
T-F-08	玩家死亡	重启面板
T-F-09	元游戏进战斗	计时启动
T-F-10	Boss 节点	Boss 胜利时长

7.4 性能测试

在 Chromium、1920×1080 下录制 60 秒战斗，对比无池/有池/池 +Worker 的平均 FPS 与 P95 帧时间（终稿填入实测值）。Cantrell 等通过对比真实疏散事件验证模型 [5]，本项目性能测试侧重帧率稳定性而非人群标定。

表 7.2: 性能对比实验（待填入实测数据）

配置	平均 FPS	P95 (ms)	备注
基线	待测	待测	无池、无 Worker
仅对象池	待测	待测	
池 + Worker	待测	待测	实体数达标

7.5 跑图与 Worker 一致性

对 RunMapGenerator 多次随机种子断言 Boss 可达；对比 Worker 开/关上怪物轨迹（允许一帧延迟），确保逻辑未漂移。

7.6 测试结论

功能覆盖战斗、元游戏、跑图、技能装配与升级闭环；单元脚本通过表明核心契约稳定。性能数据待填；部分 Effect 尚无独立 VFX，法杖完整 UI 为后续工作。

7.7 本章小结

本章给出测试策略、用例与性能实验框架，终稿须补充实测数据与系统截图。

第八章 总结与展望

8.1 工作总结

本文完成了 **Survivor And Fight** 在 LayaAir 3.x 与 TypeScript 上的 Roguelite 生存战斗游戏之分析、设计与实现。主要工作包括：轻量 ECS 玩法内核；配表驱动技能与子弹组合效果；对象池与 Web Worker 性能优化；元游戏、DAG 跑图与 MVC 界面；测试验证及社会影响讨论。

研究过程中参考了游戏引擎架构理解方法 SyDRA[19]、经典引擎架构论述 [7]，以及 Unity 物理人群疏散建模实践 [5]，将模块化、物理仿真与验证思想迁移到 H5 教学原型中。系统可演示完整游戏循环，达到毕业设计预期。

8.2 不足与展望

1. 法杖三槽完整 UI 与链式求值界面待完善；
2. ECS 可迁移至 archetype 存储以提升同屏规模 [18]；
3. 碰撞可引入空间哈希等 broad phase；
4. 补充端到端自动化测试 [15]；
5. 局外存档与元进度；
6. 美术特效与音效完善。

8.3 社会、健康与文化影响

本游戏为虚构战斗题材，应标注适龄提示并避免过度暴力表现。长时间游玩需注意视觉疲劳；Tab 暂停提供局内休息点。教学原型不采集用户敏感信息。

8.4 本章小结

本章总结全文并展望后续工作。本项目展示了在 H5 环境下结合 ECS、数据驱动与引擎性能手段构建 Roguelite 原型的可行路径。

致 谢

在本论文完成之际，谨向指导教师致以衷心感谢。老师在选题、系统架构与论文撰写方面给予了耐心指导。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试与试玩中提供的帮助；感谢家人在毕业设计期间的理解与支持。

由于本人水平有限，文中疏漏之处恳请各位老师批评指正。

参考文献

- [1] Vartika Agrahari and Sridhar Chimalakonda. What’s inside unreal engine? - a curious gaze! In *14th Innovations in Software Engineering Conference*, pages 1–5. ACM, 2021.
- [2] Ahmed Baabad, Hazura Binti Zulzalil, Sa’adah Hassan, and Salmi Binti Baharom. Software architecture degradation in open source software: A systematic literature review. *IEEE Access*, 8:173681–173709, 2020.
- [3] M. Bott, S. R. Musse, L. P. de Oliveira, and B. E. Bodmann. Implementing a physics based model of crowd movement using unreal development kit. *Journal of Gaming & Virtual Worlds*, 6(3):275–296, 2014.
- [4] L. C. Briand, C. Bunse, and J. W. Daly. A controlled experiment for evaluating quality guidelines on the maintainability of object-oriented designs. *IEEE Transactions on Software Engineering*, 27(6):513–530, 2001.
- [5] Walter Alan Cantrell, Mikel D. Petty, Samantha L. Knight, and Whitney K. Schueler. Physics-based modeling of crowd evacuation in the Unity game engine. *International Journal of Modeling, Simulation, and Scientific Computing*, 9(4):1850029, 2018.
- [6] M. Goslin and M. R. Mine. The Panda3D graphics engine. *Computer*, 37(10):112–114, 2004.
- [7] Jason Gregory. *Game Engine Architecture*. CRC Press, Boca Raton, 3 edition, 2018.
- [8] Werner Heijstek, Thomas Kühne, and Michel R. V. Chaudron. Experimental analysis of textual and graphical representations for software architecture design.

- In *2011 International Symposium on Empirical Software Engineering and Measurement*, pages 167–176. IEEE, 2011.
- [9] Dirk Helbing, Illés Farkas, and Tamás Vicsek. Simulating dynamical features of escape panic. *Nature*, 407:487–490, 2000.
- [10] F. D. McKenzie, M. D. Petty, P. A. Kruszewski, et al. Integrating crowd-behavior modeling into military simulation using game technology. *Simulation & Gaming*, 39(1):10–38, 2008.
- [11] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253, 2017.
- [12] James Munro, Cornelia Boldyreff, and Andrea Capiluppi. Architectural studies of games engines – the quake series. In *2009 International IEEE Consumer Electronics Society’s Games Innovations Conference*, pages 246–255. IEEE, 2009.
- [13] William L. Oberkamp and Christopher J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, Cambridge, UK, 2010.
- [14] Nuria Pelechano, Jan Allbeck, and Norman Badler. *Virtual Crowds: Methods, Simulation, and Control*. Morgan and Claypool, San Rafael, 2008.
- [15] Mikel D. Petty. Verification, validation, and accreditation. In *Modeling and Simulation Fundamentals*, pages 325–372. John Wiley & Sons, 2010.
- [16] Craig W. Reynolds. Flocks, herds, and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, pages 25–34. ACM, 1987.
- [17] Daniel Thalmann and Soraia R. Musse. *Crowd Simulation*. Springer, London, 2007.
- [18] Gabriel C. Ullmann, Yann-Gaël Guéhéneuc, Fabio Petrillo, Nicolas Anquetil, and Cristiano Politowski. Visualising game engine subsystem coupling patterns. In *Entertainment Computing – ICEC 2023*, volume 14455 of *Lecture Notes in Computer Science*, pages 263–274. Springer, 2023.

- [19] Gabriel C. Ullmann, Yann-Gaël Guéhéneuc, Fabio Petrillo, Nicolas Anquetil, and Cristiano Politowski. SyDRA: An approach to understand game engine architecture. *Entertainment Computing*, 52:100832, 2025.
- [20] Unity Technologies. Unity manual. <https://docs.unity3d.com/Manual/index.html>, 2016. Accessed 2026-05-17.

附录 A 名词术语及缩略词

术语/缩略语	含义
ECS	Entity Component System, 实体组件系统
Roguelite	轻 Roguelike, 含元进度与单局随机
DAG	Directed Acyclic Graph, 有向无环图
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件
Worker	Web Worker, 浏览器后台线程

附录 B 核心配表字段示例（节选）

Character 表： id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。

Bullet 表： id、prefabPath、duration、speed、damage、penetration。

SkillEffect 表： id、effect、damage、params、iconPath。

完整 JSON 见项目 docs/config/ 目录。